

LLD Case Study 6: Distributed Inventory Management System

Target Patterns: Observer Pattern, Strategy Pattern, State Pattern

Key Engineering Focus: Stock level invariants, multi-warehouse fulfillment strategies, automatic replenishment alerts, and order lifecycle states.

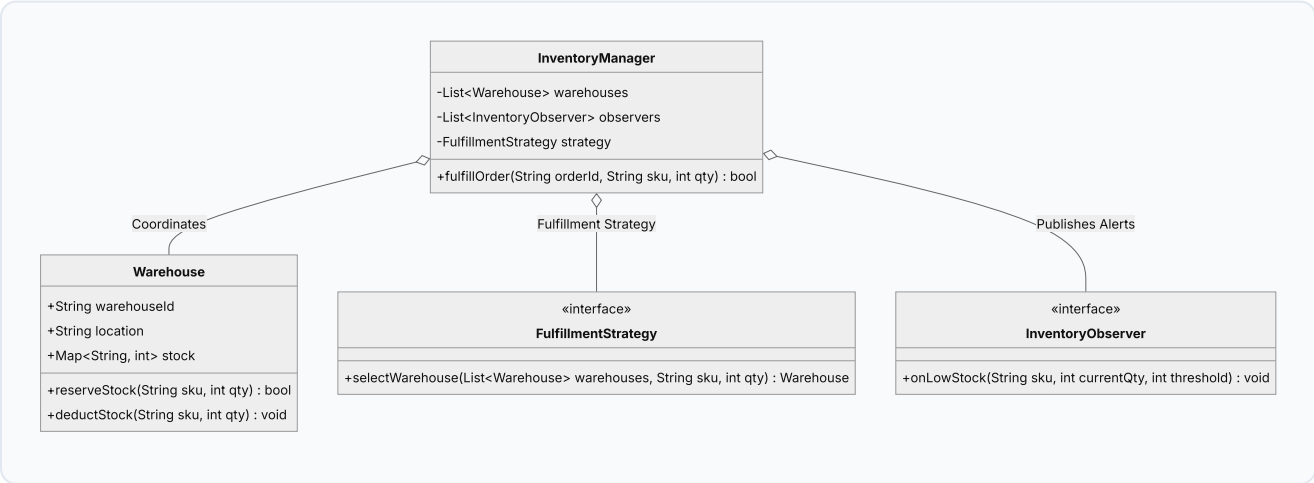
1. Problem Statement & Functional Requirements

Design an enterprise inventory management backend supporting multi-warehouse fulfillment and low-stock alerts.

Requirements:

- 1. Multi-Warehouse Fulfillment (Strategy Pattern):** Route orders to warehouses based on customizable strategies (e.g. Nearest Warehouse First vs Consolidate into Single Shipment).
- 2. Low Stock Alerts (Observer Pattern):** When stock for a SKU drops below its threshold, automatically trigger replenishment purchase orders.
- 3. Order Inventory State Machine (State Pattern):** Orders transition from `Draft` → `StockReserved` → `Dispatched` → `Delivered` (or `Released` if canceled).
- 4. Thread-Safe Inventory Mutations:** Prevent overselling under high concurrency.

2. Architecture & UML Class Diagram



3. Production-Grade Python Implementation

